

TPLA: Tensor Parallel Latent Attention for Efficient Disaggregated Prefill & Decode Inference

Xiaojuan Tang*

Fanxu Meng*

xjtang0920@gmail.com

fxmeng@stu.pku.edu.cn

Institute for Artificial Intelligence,
Peking University
Beijing, China

Pingzhi Tang

Institute for Artificial Intelligence,
Peking University
Beijing, China
stanleytang@stu.pku.edu.cn

Yuxuan Wang

Institute for Artificial Intelligence,
Peking University
Beijing, China
wangyuxuanbilly@bit.edu.cn

Di Yin

Tencent Youtu Lab
Shanghai, China
endymecyyin@tencent.com

Xing Sun

Tencent Youtu Lab
Shanghai, China
winfredsun@tencent.com

Muhan Zhang[†]

Institute for Artificial Intelligence,
Peking University
Beijing, China
muhan@pku.edu.cn

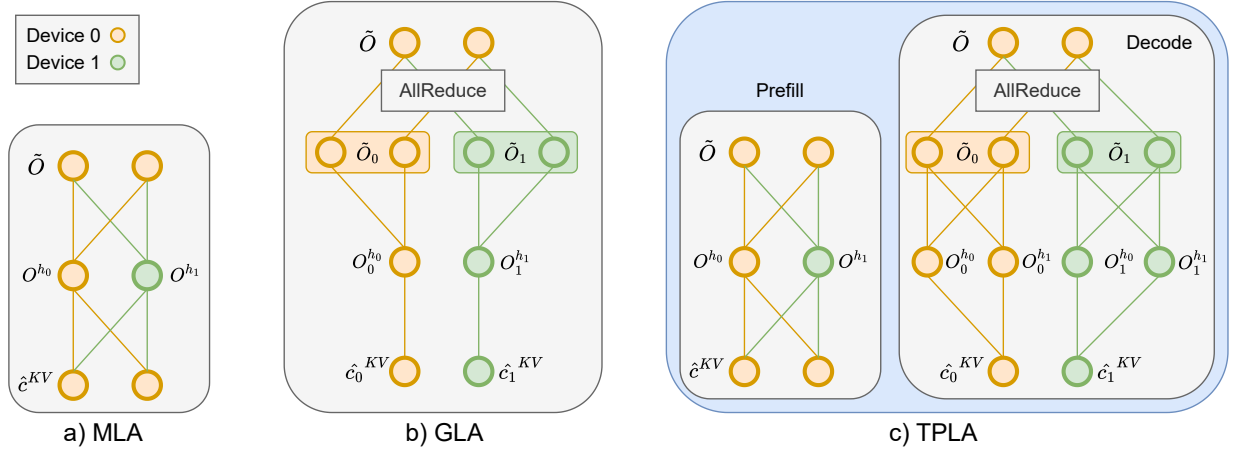


Figure 1. Comparison of MLA, GLA, and TPLA. In MLA, each device must load the entire KV cache. In GLA, each attention head only accesses the portion of the KV cache stored on its own device. In TPLA, the prefilling phase follows MLA for efficiency and accuracy, while during the decoding phase, attention heads are distributed across devices, each relying on the KV cache stored locally on its assigned device.

Abstract

Multi-Head Latent Attention (MLA), introduced in DeepSeek-V2, compresses key-value states into a low-rank latent vector $\mathbf{c}^{KV}$, caching only this vector to reduce memory. In tensor parallelism (TP), however, attention heads are computed across

multiple devices, and each device must load the full $\mathbf{c}^{KV}$, eroding the advantage of MLA over Grouped Query Attention (GQA). We present **TPLA**, a scheme that partitions both the latent representation and each head’s input dimension across devices, performs attention independently on each shard, and aggregates the results with an all-reduce. Unlike GLA, every attention head in TPLA still attends to the full latent space, preserving MLA’s representational capacity while reducing the per-device KV cache. To make TPLA **drop-in compatible with MLA checkpoints**, we further derive orthogonal reparameterizations of RMSNorm and softmax—instantiated with Hadamard and PCA transforms—that mitigate cross-shard discrepancies when slicing latent vectors across devices. Finally, we introduce a **prefill-decode separation** scheme that keeps the MLA form during compute-bound

*Both authors contributed equally to this research.

[†]Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '26, Pittsburgh, PA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2359-9/2026/03

<https://doi.org/10.1145/3779212.3790237>

prefilling and switches to TPLA during memory-bound decoding, minimizing conversion-induced error. By reducing the per-device KV cache for DeepSeek-V3 and Kimi-K2, we achieve $1.79\times$ and $1.93\times$ speedups respectively, at a 32K-token context length while maintaining accuracy on commonsense and LongBench benchmarks. TPLA can be further implemented on top of FlashAttention-3, enabling practical end-to-end acceleration.

CCS Concepts: • Computer systems organization → Neural networks.

Keywords: MLA, GQA, GLA, TP, PD Disaggregate, KV Cache, PCA, Hadamard

ACM Reference Format:

Xiaojuan Tang, Fanxu Meng, Pingzhi Tang, Yuxuan Wang, Di Yin, Xing Sun, and Muhan Zhang. 2026. TPLA: Tensor Parallel Latent Attention for Efficient Disaggregated Prefill & Decode Inference. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26), March 22–26, 2026, Pittsburgh, PA, USA*. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3779212.3790237>

1 Introduction

Currently, large language models (LLMs) [3, 8, 35, 39, 49] are typically memory-bound (limited by memory bandwidth) rather than compute-bound (limited by floating-point operations per second, FLOPs) during inference. To address this, KV cache compression [9, 10, 29, 69] and tensor parallelism [19, 25, 33, 38, 43] have emerged as two critical techniques for enabling efficient auto-regressive decoding in LLMs. KV-cache compression reduces memory footprint by pruning, merging, sharing, or quantizing intermediate key-value states. Tensor parallelism addresses memory and compute limitations by splitting large tensors—such as weight matrices—across multiple devices, enabling intra-layer parallel computation for models that cannot fit on a single GPU. GQA [2] inherently supports both KV cache compression and tensor parallelism by grouping query heads so that all heads within a group share a common set of key and value representations, which facilitates efficient distribution across multiple devices. Both theoretical analyses and empirical results demonstrate that the representational capacity of GQA is inferior to that of MLA [15, 30]. MLA introduces a pre-trained KV cache compression strategy that achieves an excellent trade-off between computational efficiency and model performance. However, when multiple attention heads are computed in parallel across multiple devices using tensor parallelism, MLA encounters a critical limitation: each device must load the full latent vector c_{KV} , undermining the memory savings that MLA offers over GQA. For example, in LLaMA-3-70B [1], the dimension of the KV cache per token is $2 \times 8 \times 128 = 2048$, and under tensor parallelism with $TP = 4$, each device holds a partitioned KV cache of size 512. In contrast, Deepseek-V3 [28] has a fixed KV cache dimension

of $64 + 512 = 576$, which must be fully replicated on each device regardless of the parallelism degree. This results in a higher per-device KV cache memory footprint compared to GQA-based models under the same tensor parallel configuration.

GLA [51] was proposed to address the tensor parallelism limitations of MLA by dividing the attention heads and latent representations into g groups (typically $g = 2$), such that each group of heads only loads its corresponding latent representation. However, this paper identifies two key limitations of GLA: (1) the reduction in KV cache size for single device comes at the cost of decreased representational capacity for each attention head; and (2) GLA requires training from scratch, which demands significant computational resources to validate its effectiveness.

To address these challenges, we propose Tensor Parallel Latent Attention (TPLA), a method that distributes the latent representations across multiple devices. Each attention head is split across devices, followed by an all-reduce operation on the output o . TPLA offers the following advantages: 1) Each attention head utilizes the full latent representation, preserving strong representational capacity; 2) Each device only loads a partition of the KV cache, improving inference speed under tensor parallelism; 3) TPLA can directly load pre-trained DeepSeek checkpoints, which incurs only a minor accuracy drop that is easily recovered; 4) We use reparameterized MLA for prefill and TPLA for decoding, reducing prefill latency while mitigating conversion-induced degradation. 5) TPLA can be viewed as a special case of GLA with more attention heads, making it compatible with FlashAttention-3.

2 Related Works

Reducing KV-Cache Memory. Generative inference with large language models (LLMs) is often constrained by the memory footprint of the key-value (KV) cache, especially for long contexts. Several families of techniques have been explored to mitigate this burden: **token pruning/evicting** [36, 48, 54, 55, 60, 67] removes KV entries for low-importance tokens based on saliency or attention estimates; **token merging** [26, 52, 56, 65] aggregates nearby or similar tokens into a single surrogate KV representation to eliminate redundancy while retaining context; **cross-layer KV sharing/fusion** [21, 44, 53, 58, 59, 62] reuses one KV cache across adjacent layers to avoid per-layer storage; **low-rank KV compression** [7, 10, 20, 40] factorizes KV matrices into low-rank components (learned or SVD-based) to reduce dimensionality and memory; and **KV-cache quantization** [12, 17, 46, 70] stores K/V tensors at reduced numeric precision (e.g., int8 or int4), cutting memory and bandwidth with modest accuracy cost. Although effective, these approaches inevitably discard or alter information in the KV cache and can degrade model accuracy. In contrast, **TPLA** leaves the KV contents intact: it reduces the amount of cache each device must hold so

the model retains full information while alleviating memory pressure. As a result, TPLA tends to preserve accuracy better than compression-based methods.

Parallelism Strategies for Deployment. Current LLMs scale to billions of parameters; to cope with the resulting memory and compute demands, engineers adopt distributed deployment to reduce wall-clock latency and time costs. **Data parallelism** [13, 42] partitions input data across the sample or batch dimension while replicating model parameters across devices. However, for very large models full replication becomes impractical; moreover, variable sequence lengths introduce load imbalance (“bubbles”) that waste compute resources. **Pipeline parallelism** [24, 34] partitions the model into contiguous blocks of layers, each placed on a different device. Intermediate activations and gradients are communicated between stages to complete the forward and backward passes, reducing cross-node traffic. This staging overlaps computation across devices to increase throughput, but pipeline bubbles can still leave some devices idle. **Tensor parallelism** [37, 47, 66] splits linear layers along their row or column dimensions, sharding tensors across devices and performing distributed matrix–matrix multiplication with collective communication. It achieves optimal performance on systems where GPUs are fully interconnected via NVLink. TPLA leverages the strengths of TP while addressing MLA’s inability to reduce the KV cache under TP. Long sequences inflate the memory footprint of intermediate activations. **Sequence/Context parallelism** [45] mitigates this by replicating the model across devices and splitting inputs along the sequence dimension so that each device processes only a subsequence. Recent long-context systems further extend this idea by *sharding the KV cache along the sequence axis* via load-balanced token partitioning and ring-style attention, and by combining GPU execution on recent tokens with CPU-assisted sparse attention over long-range, offloaded tokens [16, 57]. In contrast, TPLA shards MLA’s latent KV cache along the *feature axis*; since the sequence and feature axes are orthogonal, TPLA is naturally complementary to context-parallel and offloading-based methods, further reducing per-device KV memory and inter-device KV communication. Moreover, concurrent analysis [61] shows that MLA (with layer reordering) and MoE can shift inference from memory-bound toward a more compute-balanced regime under large batches and high-bandwidth interconnects; TPLA complements this direction by further lowering per-device KV memory load and KV bandwidth, directly targeting the remaining KV bottlenecks. **Prefill/Decode Separation** [18, 64, 68] refines sequence parallelism for LLM inference: the prefilling phase is compute-intensive and thus compute-bound, whereas the decoding phase has low per-token compute but frequent memory accesses and is memory-bandwidth-bound. To match these characteristics, different machine counts and architectures are used

across the two phases to improve latency and throughput. In TPLA, we further employ different model structures across phases—MLA during prefill to preserve accuracy while reducing computation (improving latency), and TPLA during decoding to reduce memory traffic and increase throughput.

3 Preliminary

3.1 Multi-Head Latent Attention

MLA is designed to reduce memory bandwidth overhead by compressing the Key-Value (KV) cache. Specifically, the multi-head keys and values are compressed into a single low-rank latent representation of dimension $4d_h$, denoted as $\mathbf{c}^{\text{KV}}$. Instead of reconstructing full-size keys and values from this latent representation, MLA adopts a more efficient decoding strategy. By isolating the Rotary Position Embedding (RoPE) operation, the up-projection matrix can be absorbed into the query activations, yielding Q . Similarly, the value projection is absorbed into the output projection matrix, resulting in W^{VO} (See Section 3.2). This allows for direct attention computation between Q and the normalized latent cache $\hat{\mathbf{c}}^{\text{KV}}$, followed by a projection through W^{VO} to produce the final output $\tilde{O}$. For simplicity in this initial description, we omit the RoPE components. The core computation is as follows:

$$\begin{aligned} \mathbf{c}^{\text{KV}} &\in \mathbb{R}^{B \times L \times 4d_h}, & \hat{\mathbf{c}}^{\text{KV}} &= \text{RMSNorm}(\mathbf{c}^{\text{KV}}) \in \mathbb{R}^{B \times L \times 4d_h}, \\ Q &\in \mathbb{R}^{B \times 1 \times h_q \times 4d_h}, & W^{VO} &\in \mathbb{R}^{(h_q \cdot 4d_h) \times D}, \\ O &= \text{softmax}\left(\frac{Q (\hat{\mathbf{c}}^{\text{KV}})^\top}{\sqrt{d_h}}\right) \hat{\mathbf{c}}^{\text{KV}} \in \mathbb{R}^{B \times 1 \times h_q \times 4d_h}, & (1) \\ \tilde{O} &= O W^{VO} \in \mathbb{R}^{B \times 1 \times D}. & (2) \end{aligned}$$

3.2 Matrix Absorption

Considering we apply orthogonal transformations U to reparameterize weight matrices, which involves matrix absorption. To make this process intuitive, we here present the complete calculation pipeline of MLA and show how the absorbed matrices from Section 3.1 are derived.

As stated in Section 3.1, MLA saves KV cache by multiplying the low-rank compress matrix $W^{DKV} \in \mathbb{R}^{D \times 4d_h}$ with the input sequence $X \in \mathbb{R}^{B \times L \times D}$ to obtain low-rank latent features $\mathbf{c}^{\text{KV}}$. Then, it uses the matrices $W^{UK}, W^{UV} \in \mathbb{R}^{4d_h \times (h_q \cdot d_h)}$ to derive the full-heads key $\mathbf{k}$ and value $\mathbf{v}$. Additionally, MLA also can decompose $W^Q \in \mathbb{R}^{D \times (h_q \cdot d_h)}$ to $W^{DQ} \in \mathbb{R}^{D \times r_q}$ and $W^{UQ} \in \mathbb{R}^{r_q \times (h_q \cdot d_h)}$, which reduces the activation memory during training. For positional embedding, MLA uses a decoupled RoPE strategy that uses additional multi-head queries $\mathbf{q}^{\text{PE}}$ and a shared key $\mathbf{k}^{\text{PE}}$, generated by $W^{QR} \in \mathbb{R}^{r_q \times (h_q \cdot d_r)}$ and $W^{KR} \in \mathbb{R}^{D \times d_r}$, to carry the rotary positional embeddings. The final attention output $\tilde{O}$ is computed by separately combining the non-positional part ($\mathbf{q} \mathbf{k}^\top$) and positional part ($\mathbf{q}^{\text{PE}} (\mathbf{k}^{\text{PE}})^\top$), followed by projection with

$$W^O \in \mathbb{R}^{(h_q \cdot d_h) \times D}.$$

$$\begin{aligned} \mathbf{c}^{\text{KV}} &= XW^{\text{DKV}}, \quad \mathbf{c}^{\text{Q}} = XW^{\text{DQ}}, \quad \hat{\mathbf{c}}^{\text{KV}} = \text{RMSNorm}(\mathbf{c}^{\text{KV}}), \\ \mathbf{q} &= \mathbf{c}^{\text{Q}} W^{\text{UQ}}, \quad \mathbf{k} = \hat{\mathbf{c}}^{\text{KV}} W^{\text{UK}}, \quad \mathbf{v} = W^{\text{UV}} \hat{\mathbf{c}}^{\text{KV}}, \\ \mathbf{q}^{\text{PE}} &= \text{RoPE}(\mathbf{c}^{\text{Q}} W^{\text{QR}}), \quad \mathbf{k}^{\text{PE}} = \text{RoPE}(XW^{\text{KR}}), \\ O &= \text{softmax}\left(\frac{\mathbf{q}\mathbf{k}^\top + \mathbf{q}^{\text{PE}}(\mathbf{k}^{\text{PE}})^\top}{\sqrt{d_h + d_r}}\right) \mathbf{v}, \quad \tilde{O} = O W^O. \end{aligned} \quad (3)$$

In Equation 3, the RoPE component is explicitly isolated, allowing us to restructure the attention computation using associativity of matrix multiplication. For clarity, we can temporarily omit positional encoding components and the scaling factor.

$$\begin{aligned} O W^O &= \text{softmax}(\mathbf{q}\mathbf{k}^\top) \mathbf{v} W^O \\ &= \text{softmax}(\mathbf{q}(\hat{\mathbf{c}}^{\text{KV}} W^{\text{UK}})^\top) \hat{\mathbf{c}}^{\text{KV}} W^{\text{UV}} W^O \\ &= \text{softmax}(Q(\hat{\mathbf{c}}^{\text{KV}})^\top) \hat{\mathbf{c}}^{\text{KV}} W^{\text{VO}}. \end{aligned} \quad (4)$$

Here, the matrix W^{UK} can be absorbed into $\mathbf{q}$ to derive Q in Equation 2. Similarly, the matrix W^{UV} can be absorbed into W^O . In practice, however, W^{UV} is typically not absorbed into W^O to avoid generating an impractically large matrix.

3.3 Grouped Latent Attention

During tensor-parallel decoding, MLA replicates its single latent head on every device, resulting in high KV-cache memory load across all devices. GLA avoids this replication by partitioning the latent KV cache itself. Consider a two-way tensor-parallel configuration. The latent KV cache is divided into two shards, $\mathbf{c}_0^{\text{KV}}$ and $\mathbf{c}_1^{\text{KV}}$, each assigned to one GPU. Simultaneously, attention heads h_q are grouped such that the absorbed query projection matrix Q and output projection matrix W^{VO} are partitioned along both the head dimension (h_q) and the feature dimension ($4d_h$), yielding four groups. Thus, GPU 0 operates on $(\mathbf{c}_0^{\text{KV}}, Q_{0,0}, W_{0,0}^{\text{VO}})$, while GPU 1 operates on $(\mathbf{c}_1^{\text{KV}}, Q_{1,1}, W_{1,1}^{\text{VO}})$. Each GPU independently computes its local attention output, denoted $\tilde{O}_0$ and $\tilde{O}_1$, respectively. The final output is obtained via an AllReduce operation that sums the local outputs across devices. However, this grouping imposes a structural limitation: each latent slice (width $2d_h$) is paired with only $h_q/2$ query heads, so the off-diagonal blocks $Q_{1,0}$ and $Q_{0,1}$ are never used, eliminating cross-group query-latent interactions.

$$\begin{aligned} \mathbf{c}_0^{\text{KV}}, \mathbf{c}_1^{\text{KV}} &\in \mathbb{R}^{B \times L \times 2d_h}, \quad \begin{cases} \hat{\mathbf{c}}_0^{\text{KV}} = \text{RMSNorm}(\mathbf{c}_0^{\text{KV}}) \in \mathbb{R}^{B \times L \times 2d_h}, \\ \hat{\mathbf{c}}_1^{\text{KV}} = \text{RMSNorm}(\mathbf{c}_1^{\text{KV}}) \in \mathbb{R}^{B \times L \times 2d_h}, \end{cases} \\ Q_{i,j \in \{0,1\}} &\in \mathbb{R}^{B \times 1 \times \frac{h_q}{2} \times 2d_h}, \quad \begin{pmatrix} Q_{0,0} & Q_{0,1} \\ Q_{1,0} & Q_{1,1} \end{pmatrix} = Q, \\ W_{i,j \in \{0,1\}}^{\text{VO}} &\in \mathbb{R}^{\left(\frac{h_q}{2} \cdot 2d_h\right) \times D}, \quad W^{\text{VO}} = \begin{pmatrix} W_{0,0}^{\text{VO}} & W_{0,1}^{\text{VO}} \\ W_{1,0}^{\text{VO}} & W_{1,1}^{\text{VO}} \end{pmatrix}, \end{aligned}$$

$$\begin{aligned} O_0 &= \text{softmax}\left(\frac{Q_{0,0}(\hat{\mathbf{c}}_0^{\text{KV}})^\top}{\sqrt{d_h}}\right) \hat{\mathbf{c}}_0^{\text{KV}} \in \mathbb{R}^{B \times 1 \times \frac{h_q}{2} \times 2d_h}, \\ O_1 &= \text{softmax}\left(\frac{Q_{1,1}(\hat{\mathbf{c}}_1^{\text{KV}})^\top}{\sqrt{d_h}}\right) \hat{\mathbf{c}}_1^{\text{KV}} \in \mathbb{R}^{B \times 1 \times \frac{h_q}{2} \times 2d_h}, \\ \tilde{O}_0 &= O_0 W_{0,0}^{\text{VO}} \in \mathbb{R}^{B \times 1 \times D}, \quad \tilde{O}_1 = O_1 W_{1,1}^{\text{VO}} \in \mathbb{R}^{B \times 1 \times D}, \\ O &= \text{AllReduce}(\tilde{O}_0 + \tilde{O}_1) \in \mathbb{R}^{B \times 1 \times D}. \end{aligned} \quad (5)$$

4 Tensor Parallel Latent Attention (TPLA)

Motivated by the hardware efficiency of GLA, we retain its core principle of distributing latent KV across GPUs to mitigate memory wastage and communication overload. However, directly translating an existing MLA-based model to GLA incurs a significant accuracy penalty, as shown in Figure 2. This degradation stems from a key limitation in standard GLA: the latent vector within each group only accesses half of the query heads, restricting the model's expressive power and leading to suboptimal downstream accuracy. Moreover, training a new GLA model from scratch requires a substantial cost. To address this, we further propose Tensor-Parallel Latent Attention (TPLA). Unlike standard GLA, TPLA partitions latent vectors into two groups while preserving full query heads visibility. Specifically,

$$\begin{aligned} \mathbf{c}_0^{\text{KV}}, \mathbf{c}_1^{\text{KV}} &\in \mathbb{R}^{B \times L \times 2d_h}, \quad \begin{cases} \hat{\mathbf{c}}_0^{\text{KV}} = \text{RMSNorm}(\mathbf{c}_0^{\text{KV}}) \in \mathbb{R}^{B \times L \times 2d_h}, \\ \hat{\mathbf{c}}_1^{\text{KV}} = \text{RMSNorm}(\mathbf{c}_1^{\text{KV}}) \in \mathbb{R}^{B \times L \times 2d_h}, \end{cases} \\ Q_0, Q_1 &\in \mathbb{R}^{B \times 1 \times h_q \times 2d_h}, \quad (Q_0, Q_1) = Q, \\ W_0^{\text{VO}}, W_1^{\text{VO}} &\in \mathbb{R}^{(h_q \cdot 2d_h) \times D}, \quad (W_0^{\text{VO}}, W_1^{\text{VO}}) = W^{\text{VO}}, \\ O_0 &= \text{softmax}\left(\frac{Q_0(\hat{\mathbf{c}}_0^{\text{KV}})^\top}{\sqrt{d_h}}\right) \hat{\mathbf{c}}_0^{\text{KV}} \in \mathbb{R}^{B \times 1 \times h_q \times 2d_h}, \\ O_1 &= \text{softmax}\left(\frac{Q_1(\hat{\mathbf{c}}_1^{\text{KV}})^\top}{\sqrt{d_h}}\right) \hat{\mathbf{c}}_1^{\text{KV}} \in \mathbb{R}^{B \times 1 \times h_q \times 2d_h}, \quad (6) \\ \tilde{O}_0 &= O_0 W_0^{\text{VO}} \in \mathbb{R}^{B \times 1 \times D}, \quad \tilde{O}_1 = O_1 W_1^{\text{VO}} \in \mathbb{R}^{B \times 1 \times D}, \\ O &= \text{AllReduce}(\tilde{O}_0 + \tilde{O}_1) \in \mathbb{R}^{B \times 1 \times D}. \end{aligned} \quad (7)$$

This design ensures each latent vector attends to all query heads, largely eliminating the accuracy degradation. The remaining gap is due only to tensor-parallel partitioning effects in RMSNorm and softmax operations. Through carefully designed mathematical reparameterization, TPLA can restore near-MLA accuracy. For illustration, we consider the case where the tensor-parallel degree of latent attention is 2, though the approach naturally scales to higher degrees.

4.1 RMSNorm Slicing

In MLA-like models, the “kv_a_layernorm” module normalizes input vectors using the Root Mean Square (RMS) value. Given an input vector $\mathbf{x} \in \mathbb{R}^d$ (e.g., $d = 4d_h$), the RMSNorm

is computed as:

$$\begin{aligned} \text{RMS}(\mathbf{x}) &= \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon} \\ &= \sqrt{\frac{1}{d} \|\mathbf{x}\|_2^2 + \epsilon}, \end{aligned} \quad (8)$$

$$\begin{aligned} \text{RMSNorm}(\gamma, \mathbf{x}) &= \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \odot \gamma \\ &= \text{RMSNorm}(\mathbf{1}, \mathbf{x}) \odot \gamma, \end{aligned} \quad (9)$$

where ϵ is a small constant for numerical stability; $\gamma \in \mathbb{R}^d$ is a learned scaling parameter and $\odot$ denotes element-wise multiplication.

However, we face the following challenge when applying this to tensor-parallel processing of latent attention: When input latent vector $\mathbf{x} \in \mathbb{R}^d$ is split into two partitions, $\mathbf{x}^{(0)} \in \mathbb{R}^{d/2}$ and $\mathbf{x}^{(1)} \in \mathbb{R}^{d/2}$, across different devices, the RMS computation on each local device uses only half the original dimension ($d/2$), while the true normalization requires the full $\text{RMS}(\mathbf{x})$ over dimension d .

To resolve this discrepancy, we introduce an orthogonal transformation $U \in \mathbb{R}^{d \times d}$ ($U U^\top = \mathbf{I}$) to reparameterize this module. Before introducing the conditions that this transformation U need satisfy, we first establish that RMSNorm can, in principle, be realized in a mathematically equivalent form under any orthogonal transformation.

Proposition 1.

$$\text{RMSNorm}(\mathbf{1}, \mathbf{c}) = \text{RMSNorm}(\mathbf{1}, \mathbf{c} U) U^\top \quad (10)$$

Proof. we first represent the RMSNorm process as matrix multiplication. Let $\mathbf{c} \in \mathbb{R}^{L \times d}$ be the input latent vector (for simplicity, we omit the batch size), we obtain:

$$\text{RMSNorm}(\gamma, \mathbf{c}) = \text{RMSNorm}(\mathbf{1}, \mathbf{c}) W_\gamma, \quad (11)$$

$$= D_c \mathbf{c} W_\gamma, \quad (12)$$

where D_c is a diagonal matrix of size $L \times L$ with the reciprocal of the RMS values on the diagonal and W_γ is also a diagonal matrix of size $d \times d$ with each learnable scaling parameter:

$$D_c = \text{diag} \left(\frac{1}{\text{RMS}(c_1)}, \frac{1}{\text{RMS}(c_2)}, \dots, \frac{1}{\text{RMS}(c_L)} \right), \quad (13)$$

$$W_\gamma = \text{diag} (\gamma_1, \gamma_2, \dots, \gamma_d). \quad (14)$$

Since the orthogonal transformation preserves the norm ($\|\mathbf{c} U\|_2^2 = \|\mathbf{c}\|_2^2$), we easily have $\text{RMS}(\mathbf{c}) = \text{RMS}(\mathbf{c} U)$, i.e., $D_{\mathbf{c} U} = D_c$. Thus, we can have:

$$\text{RMSNorm}(\gamma, \mathbf{c} U) U^\top = D_c \mathbf{c} U W_\gamma U^\top. \quad (15)$$

Matrix multiplication does not satisfy the commutative property. Therefore, when and only when $W_\gamma = \mathbf{I}$, we can further

prove:

$$\begin{aligned} \text{RMSNorm}(\mathbf{1}, \mathbf{c} U) U^\top &= D_c \mathbf{c} U \mathbf{I} U^\top \\ &= D_c \mathbf{c} \\ &= \text{RMSNorm}(\mathbf{1}, \mathbf{c}). \end{aligned} \quad (16)$$

□

Give by Equation 4, Equation 11 and Proposition 1, we can absorb W_γ into up-projection matrix $W^{UKV} = (W^{UK}, W^{UV})$ to achieve the $\gamma = \mathbf{1}$, ensuring the orthogonal transformations U to $\mathbf{c}$ with keeping the RMSNorm value no change. In addition, the $U^\top$ can be further absorbed into W^{UKV} ; U can be absorbed into W^{DKV} , yielding the reparameterized weight matrix:

$$W_{new}^{UKV} = U^\top W_\gamma W^{UKV}, \quad W_{new}^{DKV} = W^{DKV} U. \quad (17)$$

We have proved that any transformation U can ensure the equivalence of RMSNorm . Now we will define some conditions that serve as the computational basis for U , deferring the specific calculation method to a later section.

Condition 1 (RMSNorm Slicing Condition).

$$\alpha \|(\mathbf{c} U)_0\|_2^2 \approx \beta \|(\mathbf{c} U)_1\|_2^2 \approx \|\mathbf{c} U\|_2^2 = \|\mathbf{c}\|_2^2. \quad (18)$$

Here, α and β are fixed constants, invariant to changes in the input data distribution (How to calculate their specific values is detailed in Section 4.3). $(\mathbf{c} U)_1$ and $(\mathbf{c} U)_2$ are the two partitions of the transformed $\mathbf{c}$ split across devices. By satisfying this, the new RMS values computed from the two partitions are proportional to the global value, thereby providing an accurate approximation of the global RMSNorm :

$$\begin{aligned} \text{RMS}(\mathbf{c}) &= \sqrt{\frac{1}{d} \|\mathbf{c}\|_2^2 + \epsilon} \\ &\approx \sqrt{\frac{\alpha}{d} \|(\mathbf{c} U)_0\|_2^2 + \epsilon} \\ &\approx \sqrt{\frac{\alpha}{2}} \text{RMS}((\mathbf{c} U)_0) \approx \sqrt{\frac{\beta}{2}} \text{RMS}((\mathbf{c} U)_1). \end{aligned} \quad (19)$$

Thus, we can compute RMSNorm in a tensor-parallel manner while maintaining the integrity of the normalization process.

4.2 Softmax Slicing

In common tensor-parallel techniques, matrices are typically split across devices to perform either row or column parallelism. For our TPLA attention score computation, row parallelism is employed, where the weight matrix A is split across devices according to its rows. To ensure a valid matrix multiplication, the input matrix X is correspondingly partitioned column-wise into X_1 and X_2 , such that

$$XA = (X_1 \ X_2) \begin{pmatrix} A_1 \\ A_2 \end{pmatrix} = X_1 \cdot A_1 + X_2 \cdot A_2 = Y_1 + Y_2 = Y \quad (20)$$

where X_1 and A_1 are computed on GPU 0 to produce Y_1 , and X_2 and A_2 are computed on GPU 1 to produce Y_2 . The final output Y is then all-reduced by summing Y_1 and Y_2 .

In the context of softmax computation (Equation 3), TPLA partitions $\mathbf{c}^{\text{KV}}$ and ensures that the computation of positional components remains unaffected. Specifically, the shard of the key positional embedding $\mathbf{k}^{\text{PE}}$ must be replicated across devices so that the local positional values remain consistent with the global values. As for non-positional parts ($Q (\hat{\mathbf{c}}^{\text{KV}})^\top$), the latent vectors $\mathbf{c}$ are split into two devices, and each device performs only its local computation, i.e., GPU 0 computes $Q_0 (\hat{\mathbf{c}}_0^{\text{KV}})^\top$, while GPU 1 computes $Q_1 (\hat{\mathbf{c}}_1^{\text{KV}})^\top$. However, in most cases,

$$\begin{aligned} & \text{softmax}(Q (\hat{\mathbf{c}}^{\text{KV}})^\top + \mathbf{q}^{\text{PE}} (\mathbf{k}^{\text{PE}})^\top) \\ &= \text{softmax}(Q_0 (\hat{\mathbf{c}}_0^{\text{KV}})^\top + Q_1 (\hat{\mathbf{c}}_1^{\text{KV}})^\top + \mathbf{q}^{\text{PE}} (\mathbf{k}^{\text{PE}})^\top) \\ &\neq \text{softmax}(Q_0 (\hat{\mathbf{c}}_0^{\text{KV}})^\top + \mathbf{q}^{\text{PE}} (\mathbf{k}^{\text{PE}})^\top) \quad [\text{GPU 0}] \\ &\neq \text{softmax}(Q_1 (\hat{\mathbf{c}}_1^{\text{KV}})^\top + \mathbf{q}^{\text{PE}} (\mathbf{k}^{\text{PE}})^\top) \quad [\text{GPU 1}] \end{aligned}$$

Thus, the challenge of TPLA is how to ensure the global value of $Q (\hat{\mathbf{c}}^{\text{KV}})^\top$ can be approximated from local computations. We first show that applying any orthogonal transformation U does not alter the equivalence of the original softmax output. Based on Equation 4, we easily have:

$$\begin{aligned} Q (\hat{\mathbf{c}}^{\text{KV}})^\top &= QU (\hat{\mathbf{c}}^{\text{KV}}U)^\top = \mathbf{q} (U^\top W^{UK})^\top (\hat{\mathbf{c}}^{\text{KV}}U)^\top \\ &= Q' (\hat{\mathbf{c}}^{\text{KV}}U)^\top \end{aligned} \quad (21)$$

Analogous to Section 4.1, by absorbing U into W^{UKV} and W^{DKV} (equivalently, into Q to obtain Q'), we can impose an orthogonal transformation U , which preserves the original softmax computation and must satisfy:

Condition 2 (Softmax Slicing Condition).

$$Q' (\hat{\mathbf{c}}^{\text{KV}}U)^\top \approx \mu Q'_0 (\hat{\mathbf{c}}^{\text{KV}}U)_0^\top \approx \nu Q'_1 (\hat{\mathbf{c}}^{\text{KV}}U)_1^\top \quad (22)$$

Accordingly, by determining the coefficients μ and ν , each device can perform its local computation and scale by the factor, thereby approximating the global value.

4.3 Reparameterization Methods

From the derivation above, we need to find one orthogonal transformation matrix U applied to projection weights, ensuring that the transformation satisfies Condition 1 and Condition 2 — local computations can accurately approximate the global RMSNorm and softmax values. To achieve this, we explore two potential methods: Hadamard Matrix Transformation and Principal Component Analysis (PCA).

4.3.1 Hadamard Matrix Transformation Hadamard matrix is a special orthogonal matrix where each entry is either +1 or -1. It operates by balancing the numbers, thereby reducing extreme numerical deviations and promoting a more uniform distribution of data. In practice, we typically use the function `scipy.linalg.hadamard(d)` to generate

a Sylvester-type Hadamard matrix (also known as a Walsh-Hadamard matrix) $H_d \in \mathbb{R}^{d \times d}$, constructed using a deterministic recursive rule:

$$H_{2n} = \begin{pmatrix} H_n & H_n \\ H_n & -H_n \end{pmatrix}, \quad H_1 = (1). \quad (23)$$

To increase robustness, a random diagonal matrix D , with entries drawn from ± 1 is multiplied with H_d , thereby breaking deterministic structure while preserving orthogonality. Since $H_d H_d^\top = d \cdot \mathbf{I}$, orthonormality is achieved by scaling H_d by $\frac{1}{\sqrt{d}}$, ensuring that normalization values are preserved.

Take an illustrative example. Consider a 4-dimensional vector $\mathbf{c} = (100, 0, 0, 0)$ and the 4×4 Hadamard matrix H_4 . The transformed vector $\mathbf{c}' = \mathbf{c}H_4$ is:

$$\mathbf{c}' = (100, 0, 0, 0) \begin{pmatrix} \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \end{pmatrix} = (50, 50, 50, 50). \quad (24)$$

When applied to an input vector $\mathbf{c}$: $\mathbf{c}' = \mathbf{c}H_d$, $d = 4d_h$, we obtain: $\frac{\|(\mathbf{c}H_d)_1\|_2^2}{d/2} \approx \frac{\|(\mathbf{c}H_d)_2\|_2^2}{d/2} \approx \frac{\|\mathbf{c}H_d\|_2^2}{d} = \frac{\|\mathbf{c}\|_2^2}{d}$, satisfies our key Condition 1 and easily determine $\alpha = 2$. This uniformity minimizes approximation error in tensor-parallel RMSNorm, validated experimentally in Figure 2.

However, satisfying Condition 2 is more challenging. While the magnitudes of the Hadamard transformed vector elements are balanced, due to the presence of both positive and negative signs, this transformation does not guarantee that the multiplication of the two parts will be approximately clear. To illustrate this, consider the following example: let $Q = (100, 0, 0, 0)$ and $\mathbf{c} = (0, 0, 80, 0)$. After applying the Hadamard transformation, we have:

$$\begin{aligned} Q' &= QH_4 = (50, 50, 50, 50), \\ \mathbf{c}' &= \mathbf{c}H_4 = (40, 40, -40, -40). \end{aligned}$$

The element-wise product is: $Q \cdot \mathbf{c}' = (200, 200, -200, -200)$. When this product is split into two parts, we get: $400 \neq -400 \neq 0$. This demonstrates that a standard Hadamard transformation cannot ensure Condition 2. A potential direction to address this issue is to search for an optimized Hadamard matrix via dimension permutations that minimizes the discrepancy between partitions. We leave the investigation of such optimized transformations for future work.

4.3.2 Principal Component Analysis (PCA) PCA is a widely used technique in statistics and machine learning for dimensionality reduction, feature extraction, etc. It transforms a dataset into a new coordinate system such that the greatest variances of the data are captured along the new axes (principal components). Each subsequent component is orthogonal to (i.e., uncorrelated with) the preceding ones. In our context, we leverage this property to project data onto orthogonal dimensions, with the eigenvalues indicating the variance captured along each eigenvector and thus reflecting

the statistical importance of each dimension. Moreover, for mean-centered features, the variance is equivalent to mean of the squared values, closely related to squared RMS value.

To implement this, we process a calibration dataset (e.g., Wikitext-2) to collect the KV latent cache (excluding position features) represented by $F \in \mathbb{R}^{(B \cdot L) \times d}$. We then compute the eigenvectors U and eigenvalues Λ by performing eigenvalue decomposition on the covariance matrix $\Sigma_F = U\Lambda U^\top$.

Based on Condition 1, we define α as proportion of variance captured by the first $d/2$ principal components. Similarly, β represents the proportion of variance captured by the remaining components. These ratios are as follows:

$$\alpha = \frac{\sum_{i=1}^{d/2} \lambda_i}{\sum_{i=1}^d \lambda_i}, \quad \beta = \frac{\sum_{i=d/2+1}^d \lambda_i}{\sum_{i=1}^d \lambda_i}. \quad (25)$$

For Condition 2, the metrics μ and ν are defined in the same manner, making them equivalent to α and β , respectively.

4.4 TPLA as a Special Case of GLA

Tensor parallelism in GLA employs a two-dimensional sharding scheme, splitting both head axis h_q and the latent dimension axis $4d_h$ across devices. For a query tensor $Q \in \mathbb{R}^{B \times L \times h_q \times 4d_h}$, this partitioning yields four logical sub-tensors:

$$Q = \begin{pmatrix} Q_{0,0} & Q_{0,1} \\ Q_{1,0} & Q_{1,1} \end{pmatrix}, \quad \text{where } Q_{i,j} \in \mathbb{R}^{B \times L \times \frac{h_q}{2} \times 2d_h}.$$

In standard GLA, they are distributed with only two devices. Thus, only two diagonal blocks can be materialized locally—one per device—without additional communication:

$$\begin{cases} \text{Device 0: } Q_{0,0} \in \mathbb{R}^{B \times L \times \frac{h_q}{2} \times 2d_h}, \\ \text{Device 1: } Q_{1,1} \in \mathbb{R}^{B \times L \times \frac{h_q}{2} \times 2d_h}. \end{cases}$$

Each latent slice (of width $2d_h$) is paired with only half of the query heads ($h_q/2$) and thus unable to access the off-diagonal head slices $Q_{1,0}$ and $Q_{0,1}$. In effect, these parts do not contribute to the computation, leading to significant accuracy degradation on downstream tasks.

In contrast, TPLA overcomes this limitation by enabling each partitioned latent vector to attend to all query heads. It achieves this by reformulating the computation to be algebraically equivalent to a GLA system with double the number of heads. Concretely, define a conceptual query tensor Q' that duplicates the original query along the head dimension:

$$Q' = \begin{pmatrix} Q_{0,0} & Q_{0,1} \\ Q_{1,0} & Q_{1,1} \\ Q_{0,0} & Q_{0,1} \\ Q_{1,0} & Q_{1,1} \end{pmatrix} \in \mathbb{R}^{B \times L \times (2h_q) \times (4d_h)},$$

For h_q original heads, TPLA's duplication additionally creates h_q heads. This is algebraically equivalent to a GLA system with $2h_q$ heads $4d_h$ latent dimension. Thus, we can perfectly follow the same as TPLA sharding way. When split into two

device, we have:

$$\begin{cases} \text{Device 0: } \begin{bmatrix} Q_{0,0} & Q_{1,0} \end{bmatrix} \in \mathbb{R}^{B \times L \times h_q \times 4d_h} \\ \text{Device 1: } \begin{bmatrix} Q_{0,1} & Q_{1,1} \end{bmatrix} \in \mathbb{R}^{B \times L \times h_q \times 4d_h} \end{cases}$$

Generalizing to k devices, let g denote the TPLA replication factor (number of latent-cache-slice groups). TPLA divides k devices into r group size of size k/g . Each group holds a disjoint slice of the latent axis of width $4d_h/g$ and replicates the complete set of head parameters. Within each group, the head axis is sharded across the k/g devices. Consequently, each device processes $\frac{h_q}{k/g}$ heads, and $\frac{4d_h}{g}$ latent features. For $g = 2, k = 2$, this recovers the two-device case above, where each device receives h_q heads and $2d_h$ latent width. The computational complexity arising from parameter replication is analyzed in Section 4.5.

In summary, because TPLA preserves GLA's sharding pattern—differing only by a constant-factor replication of head parameters—state-of-the-art attention optimizations (e.g., FlashAttention-3) can be applied to TPLA with minimal changes to the underlying framework.

4.5 Prefill-Decode Separation

Large language model inference is usually into two phases with distinct performance characteristics: *prefill* and *decode*. The prefill phase processes the entire prompt in a single, parallel pass to compute the initial Key-Value (KV) cache. This large-batch computation is fundamentally compute-bound. The subsequent decode phase autoregressively generates one token at a time. Each generation step requires reading the entire, growing KV cache from high-bandwidth memory (HBM) to on-chip SRAM. As the context length increases, this large data transfer becomes the primary bottleneck, making the decode phase memory-bound.

Our proposed technique, TPLA, addresses this challenge by reducing the KV cache size on each device. This reduction effectively alleviates the memory bandwidth bottleneck at the cost of a minor increase in computation. In essence, TPLA shifts the decode phase from being memory-bound towards being more compute-bound. Note that “more compute-bound” here does not mean introducing substantially more computation; rather, it reflects the reduced memory loads. In fact, although TPLA conceptually introduces duplication, each device's latent dimension is reduced by half, keeping the dominant compute in the main attention module unchanged relative to MLA. A detailed analysis is as below.

Complexity Analysis of TPLA. As discussed in Section 4.4, TPLA requires the replication of head-specific parameters across latent attention groups. Specifically, let's analyze the case with a tensor parallelism (TP) degree of 2. We consider a hidden state $X \in \mathbb{R}^{L_q \times D}$ for a single-batch inference using the MLA-absorbing strategy. The dominant cost lies in the attention computation. For a KV cache of length

S_{KV} , the complexity of the TPLA attention module (Equation 6) is approximately $O(L_q \times S_{KV} \times h_q \times 2d_h \times 2)$. In comparison, for MLA (Equation 1), with $TP=2$, the heads are split into two groups of $\frac{h_q}{2}$, leading to a complexity of $O(L_q \times S_{KV} \times \frac{h_q}{2} \times 4d_h \times 2)$. These two complexities are arithmetically equivalent. (Strictly speaking, positional components introduce additional overhead, since TPLA doubles the number of heads without reducing the RoPE dimension, but this effect is relatively minor.) Similarly, the $\tilde{O}$ computations are also equivalent.

Beyond the main attention computation, TPLA modifies other calculations: the computation of c^{KV} in TPLA is distributed across two devices, reducing complexity by $O(L_q \times D \times 2d_h \times 2)$, while the computation of Q increases by $O(L_q \times D \times 2h_q \times d_h)$. However, as context length grows, the overall cost is increasingly dominated by the self-attention module (see Figure 4).

To mitigate the additional computational overhead of TPLA, we also strategically decouple the attention mechanisms: retaining standard MLA during compute-intensive prefilling to minimize computation and reduce loss caused by converting MLA to TPLA, while activating TPLA exclusively during memory-bound decoding to minimize KV cache footprint. This hybrid approach thereby further optimizes performance by matching each phase to its most suitable mechanism.

Discussion. The above analysis focuses on MLA-absorbing computation. However, during training—when constructing the full-size KV cache and performing attention—TPLA doubles the number of heads while keeping the head dimension d_h unchanged. This increases the overall computational load, making training TPLA from scratch potentially expensive. In practice, since converting from MLA to TPLA via our reparameterization incurs only a small accuracy loss, post-training or alignment requires limited additional compute. That said, designing effective and efficient training strategies for TPLA remains an open problem, which we leave for future work.

5 Experiment

An advantage of TPLA over GLA [51] is that TPLA can be applied without training a model from scratch. It allows direct loading of models originally trained with MLA (e.g., the DeepSeek series [14, 15, 28], Kimi-k2 [50], TransMLA [30]), and—through our proposed reparameterization method and Prefill/Decode Separation technology—mitigates accuracy degradation caused by changes in the attention mechanism.

5.1 Accuracy on Commonsense Tasks

In this section, we evaluate TPLA by directly loading MLA checkpoints *without any additional training* on short-text

commonsense tasks. Accuracy is measured with the LightEval framework on MMLU [22], ARC (Easy/Challenge) [11], PIQA [6], HellaSwag [63], OpenBookQA (OBQA) [32], and WinoGrande (WG) [41]. Results are reported in Table 1. For **GLA**, following the procedure in Section 3.3, we partition the attention heads into two groups, assigning each group half of the latent dimension. As shown in Table 1, discarding half of each head’s KV cache causes severe accuracy degradation—for example, WikiText-2 perplexity (ppl) increases from 6.31 with MLA to 2212 with GLA—whereas **TPLA**, which allows each attention head to use the full latent dimension across different devices, maintains a ppl of 7.24. This comparison indicates that TPLA preserves MLA’s representational capacity while reducing the per-device KV-cache footprint. We therefore expect that pretraining TPLA from scratch would outperform GLA. For **TPLA**, we first use WikiText-2 [31] as a calibration set and, following Sections 4.1 and 4.2, slice the MLA components (the KV_a RMSNorm and the softmax) to obtain TPLA weights. As shown in Table 1, this requires no fine-tuning and yields only minor accuracy degradation. The reparameterization method used here is the PCA-based approach described in Section 4.3.2. For **TPLA (align)**, we use the SmolLM-Corpus [5] for lightweight alignment. First, we match the layer-wise input/output features of TPLA to those of the original MLA model using 256 random samples of length 2,048 for 10 epochs, minimizing MSE with the Muon optimizer (initial learning rate $1e-6$). Next, we align the end-to-end model outputs using 100M tokens, following the TransMLA setting (batch size = 32, learning rate = $2e-5$, warmup ratio = 0.03, cosine scheduler, max sequence length = 4096). Experiments are conducted on a node with $8 \times H20$ GPUs (96 GB per GPU, ~ 148 FP16 TFLOPS each). This small amount of alignment data is sufficient to recover the accuracy of the converted model. For **TPLA (pd sep.)**, we use MLA in the prefilling stage with the same reparameterization but *without* slicing the RMSNorm or softmax; prefilling thus behaves identically to the original MLA, and the KV cache can be partially reused by TPLA during decoding. By avoiding slicing for most tokens, this prefill-decode separation achieves accuracy close to the original model *without any training*. For **LLaMA-2-7B**, we first apply TransMLA [30] to convert MHA/GQA to MLA (64 RoPE dimensions and 512 NoPE dimensions—corresponding to a pruning ratio of 92.97%) and then fine-tune to recover accuracy. We subsequently convert the MLA checkpoint released by TransMLA directly into TPLA. With TransMLA as a bridge, TPLA can be applied to pretrained models that originally use MLA, GQA, or MHA.

These experiments demonstrate that converting MLA-based models to TPLA can effectively preserve accuracy. Given the benefits of TPLA for tensor parallelism, this presents a promising approach for efficient model deployment and acceleration.

Table 1. WikiText-2 Perplexity and Commonsense reasoning accuracy when converting the MLA to TPLA. The six benchmarks include MMLU, ARC (easy and challenge), PIQA, HellaSwag, OpenBookQA (OBQA), and Winogrande.

Model	PPL↓	Avg.↑	MMLU	ARC	PIQA	HellaSwag	WinoGrande	OBQA
DeepSeek-V2-Lite	6.31	61.75	43.19	60.39	80.20	74.46	65.43	45.80
- GLA	2212.	33.77	25.32	26.77	51.47	25.65	49.88	23.60
- TPLA	7.24	54.33	37.67	51.50	75.46	63.56	59.19	38.60
- TPLA (<i>align</i>)	6.51	61.52	42.72	62.58	79.82	73.32	65.90	44.80
- TPLA (<i>pd sep.</i>)	6.31	61.44	43.19	60.14	80.09	74.41	65.59	45.20
DeepSeek-V2	3.89	68.32	51.91	69.09	83.13	82.17	74.03	49.60
- TPLA	4.72	63.40	47.19	65.04	80.69	75.46	66.61	45.40
DeepSeek-V3	3.24	72.10	60.85	77.16	85.58	85.41	75.22	48.40
- TPLA	4.02	68.00	54.88	75.25	82.70	80.69	69.46	45.00
Kimi-K2-Base	1.91	73.52	63.20	78.75	85.47	87.55	75.93	50.20
- TPLA	2.44	70.49	57.64	76.00	83.79	83.53	72.38	49.60
LLaMA-2-7B	5.47	59.85	41.43	59.24	78.40	73.29	64.96	41.80
- TransMLA	5.88	58.95	40.38	57.64	78.18	70.59	62.90	44.00
- TPLA	6.74	54.68	36.12	53.21	74.81	64.52	59.04	40.40

Table 2. Longbench accuracy when converting the MLA to TPLA.

Model	Avg.	Multi-QA	Single-QA	Summarize	Few-Shot	Code	Synthetic
DeepSeek-V2-Lite	28.90	12.43	20.04	16.74	62.59	57.86	3.77
- TPLA	10.98	6.96	9.20	6.91	25.29	14.41	3.11
- TPLA (<i>align</i>)	22.60	10.97	14.67	16.59	58.03	31.59	3.77
- TPLA (<i>pd sep.</i>)	24.44	13.95	15.07	8.62	59.10	46.67	3.23
DeepSeek-V3	58.19	55.37	51.65	23.97	69.42	80.09	68.63
- TPLA	44.52	35.02	38.53	12.55	53.01	61.20	66.83
- TPLA (<i>pd sep.</i>)	56.04	53.01	50.17	21.39	67.22	75.97	68.47
Kimi-K2-Base	54.78	48.18	49.06	24.55	67.44	73.80	65.67
- TPLA	35.09	31.81	41.28	16.62	65.47	32.35	23.00
- TPLA (<i>pd sep.</i>)	52.39	45.33	46.95	20.75	67.71	67.64	66.00

5.2 Accuracy on Longbench Tasks

As context length grows, memory traffic increases and the KV-cache size becomes a primary driver of latency and throughput. To assess how TPLA converted from MLA behaves on long inputs, we evaluate on *LongBench* [4], a bilingual (English/Chinese), multi-task benchmark for long-context understanding that comprises 21 tasks across six categories (e.g., question answering, summarization, and few-shot learning). Because long-text inference is slower, we report results only for DeepSeek-V2-Lite and DeepSeek-V3. Due to GPU memory constraints, the maximum input context length is set to 31,500 tokens for DeepSeek-V2-Lite and 127,500 tokens for DeepSeek-V3. For each task, the output length is kept the same as in the original paper. The outcomes are summarized in Table 2. We observe that slicing errors in RMSNorm and softmax accumulate with sequence length, leading to some

degradation on LongBench. **TPLA (align)** follows the same alignment recipe as in the previous section, but its effectiveness is limited because the alignment corpus is formed by concatenating short texts. In contrast, **TPLA (pd sep.)** adopts a prefill–decode separation: MLA is used unchanged in the prefill stage (no slicing of RMSNorm/softmax), and the resulting KV cache is partially reused by TPLA during decoding, which reduces first-token latency and accuracy loss. On DeepSeek-V2-Lite, the training-free **TPLA (pd sep.)** surpasses the aligned variant, and on DeepSeek-V3 the model retains strong long-form reasoning with only a modest average drop of 2.15%. These small losses, compared with training from scratch, are likely recoverable with a small amount of additional training.

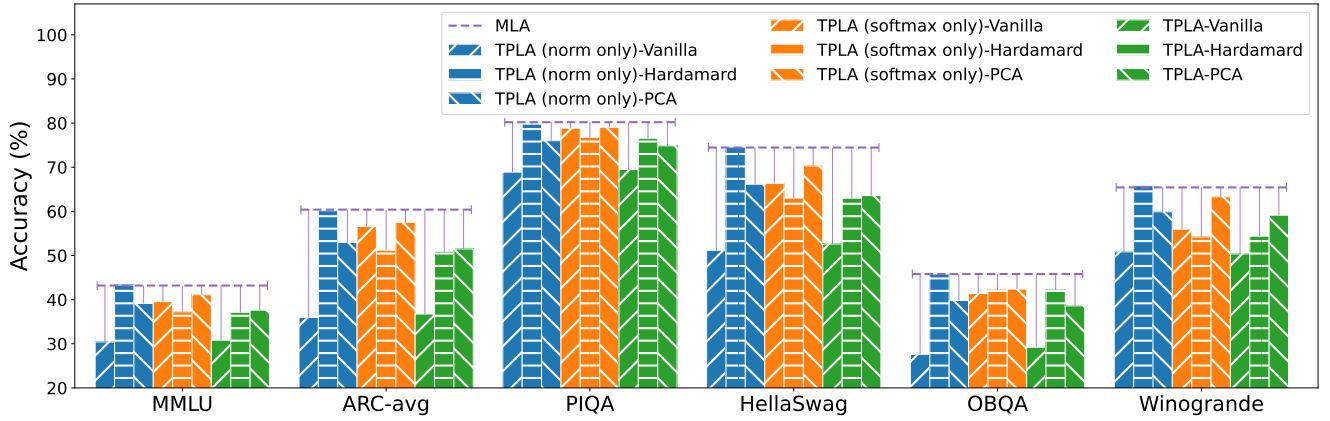


Figure 2. Ablation study (PD separation disabled). Accuracy across multiple benchmarks under different tensor-parallelism methods (colors) and reparameterization strategies (textures). The purple horizontal line marks the original DeepSeek-V2-Lite accuracy, and the vertical bars show each method’s accuracy drop relative to this MLA baseline. **TPLA (norm only)** parallelizes RMSNorm across two devices, followed by an allgather before the softmax. **TPLA (softmax only)** applies RMSNorm normally and parallelizes the softmax. **TPLA** parallelizes both RMSNorm and softmax. **Original** splits parameters evenly; **Hadamard** balances parts prior to splitting; **PCA** concentrates information into earlier dimensions before splitting.

5.3 Ablation Study

5.3.1 Part 1 We highlight two structural differences. (i) *Per-head latent capacity*: GLA gives each attention head only half of the latent dimension, whereas TPLA preserves the full latent dimension per head. (ii) *Prefill–decode (PD) separation*: during the compute-intensive prefill stage we keep the reparameterized MLA form *without* splitting RMSNorm or softmax; during decoding, TPLA uses PD separation while consuming the prefill KV cache. We analyze the results in Table 1 to quantify these effects:

1) MLA → GLA conversion. Directly converting MLA to GLA forces each attention head to access only half of its original latent representation, causing substantial information loss and a marked accuracy drop across all benchmarks.

2) Prefill–decode separation. Avoiding RMSNorm/softmax partitioning in prefill reduces approximation error for the vast majority of tokens. Moreover, the MLA reparameterization enables the prefill KV cache to be used directly by TPLA at decode time, improving both quality and efficiency.

5.3.2 Part 2 In Section 4, we identified **RMSNorm** and **softmax** as the primary sources of error when converting MLA to TPLA. To mitigate this, we proposed two reparameterization strategies, **Hadamard-based** and **PCA-based**, to reduce the accuracy degradation introduced by parallelizing these components. This section presents ablation studies analyzing the impact of each reparameterization method on individual modules. Figure 2 reports these ablations with PD separation disabled, reflecting only slicing and reparameterization effects rather than our end-to-end system. In contrast,

end-to-end results with TPLA (pd sep.) show negligible accuracy loss (Tables 1 and 2). The key findings from Figure 2 are:

1) Error ordering. Empirically, slicing *RMSNorm* incurs the least loss, slicing *softmax* is worse, and slicing both is worst:

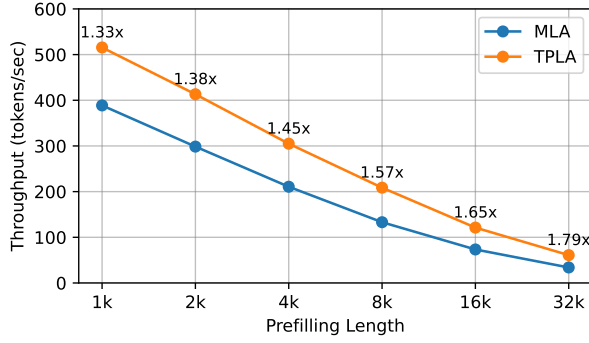
2) TP on RMSNorm only: The Hadamard-based method balances the norm computation across devices effectively, leading to accuracy comparable to the original MLA model on multiple tasks.

3) TP on softmax only: The PCA-based method concentrates information into the dimensions assigned to device 1, effectively preserving accuracy. In contrast, the Hadamard-based method fails to improve softmax accuracy. We hypothesize that the exponential nature of softmax makes it more sensitive to imbalance. Although Hadamard-based reparameterization achieves statistical balance across devices, small per-sample perturbations may result in significant asymmetries, adversely affecting final accuracy.

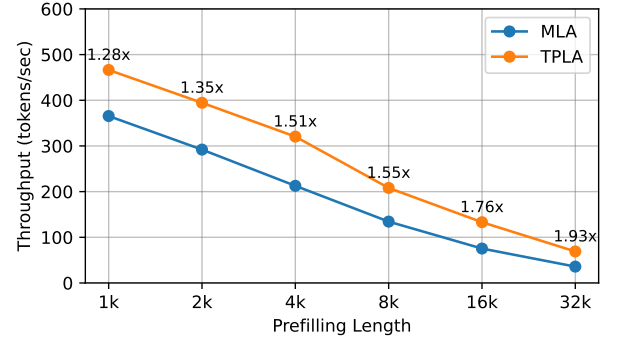
4) TP over both RMSNorm and softmax. When both components are parallelized, the PCA-based reparameterization consistently achieves the best accuracy. Consequently, we adopt this configuration for all experiments unless otherwise stated.

5.4 Inference Speedup with TPLA

5.4.1 Decoding Throughput LLM decoding is often *memory-bandwidth bound*. TPLA splits each attention head’s input dimension across two devices, reducing per-device memory traffic and alleviating the bandwidth bottleneck. We evaluate the speedup of TPLA over MLA on two large models,

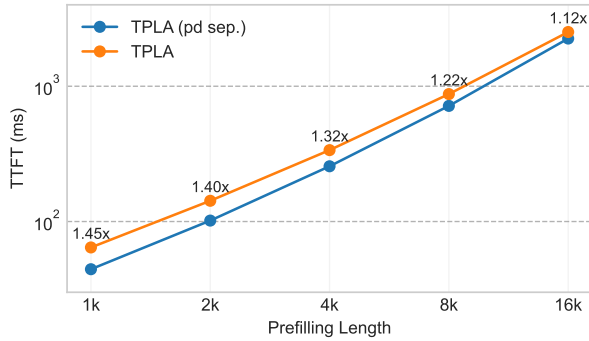


(a) H800 for DeepSeek-V3-0324

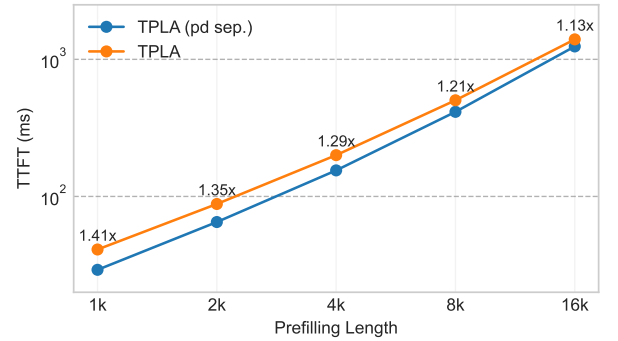


(b) H800 for Kimi-K2-Base

Figure 3. Throughput (Decoding) comparing MLA and TPLA.



(a) H800 for DeepSeek-V3-0324



(b) H800 for Kimi-K2-Base

Figure 4. Latency (TTFT) comparing TPLA and TPLA (pd sep.).

DeepSeek-V3-0324 (685B parameters) and **Kimi-K2-Base** (1T parameters). Because these models are extremely large and Mixture-of-Experts (MoE) routing can confound attention-speed effects, we remove MoE for timing. Both models are converted to **BF16**. All experiments use **FlashAttention-3** to ensure a fair comparison.

For **TPLA** with $TP=2$, the number of attention heads per device stays unchanged, while the latent dimension changes from $64+512$ to $(64+256) \times 2$, so each device holds a 320-dimensional KV cache. For **MLA** with $TP=2$, the latent dimension is unchanged and heads are distributed across devices (e.g., DeepSeek-V3: 64 heads $\times 2$; Kimi-K2: 32 heads $\times 2$). For **MLA** with $TP>2$, we continue splitting along heads only. For **TPLA** with $TP>2$, we further split heads *in addition* to halving the latent dimension; for example, with $TP=4$ on Kimi-K2-TPLA, we use 32 heads $\times 2$ per device with a 320-dimensional latent per head. In this setting, the per-device compute halves, while memory traffic matches $TP=2$; decoding remains memory-bound, so the speedup is similar to $TP=2$. Consequently, we report measurements on two H800 GPUs.

Figure 3 configures the maximum batch size at each context length. At a decoding length of 4096, **TPLA** with $2d_h$ achieves up to $\sim 2\times$ the throughput of the single-head-latent **MLA** with $4d_h$, due to the smaller per-device KV cache. Our parallelization-friendly design raises peak throughput and is resilient under adverse serving loads. At a 32k context length, **DeepSeek-TPLA** is **1.79 $\times$** faster than **MLA**, and **Kimi-K2-TPLA** is **1.93 $\times$** faster.

5.4.2 Prefilling Latency The *prefilling* stage of LLM inference is *compute-bound*. Under TPLA’s TP separation, each device retains the original number of heads, whereas MLA can reduce heads per device by splitting them across devices. As a result, the original TPLA is not ideal for the compute-bound prefill stage. To address this, we introduce **TPLA (pd sep.)**: it applies the same reparameterization to MLA but *does not* slice RMSNorm or softmax, thereby introducing no approximation error. During prefill, the structure matches MLA: under TP we do not change the latent dimension but partition heads across devices. This significantly reduces per-device compute and alleviates the compute bottleneck.

Figure 4 reports **TTFT** (Time to First Token) on two H800 GPUs for MoE-removed DeepSeek-V3-0324 and Kimi-K2-Base.

At a 1K prompt length, TPLA (pd sep.) is $1.4\times$ faster than TPLA for both models. Given its accuracy-friendly design, this $1.4\times$ gain is essentially a “free lunch.”

6 Conclusion, Limitation and Future Work

We introduce TPLA, which combines the KV cache compression efficiency of MLA with strong compatibility for Tensor Parallelism. It can directly inherit checkpoints from MLA-pretrained models. With two proposed reparameterization techniques, it substantially reduces the loss incurred by converting the attention formulation; combined with PD separation, the training-free conversion error can be driven to a very small level. We evaluate TPLA on commonsense reasoning tasks and the more challenging LongBench benchmark, finding that it preserves the original model’s accuracy well. Extensive ablations confirm the effectiveness of our TP slicing and reparameterization designs. On H800 GPUs, TPLA achieves up to $2\times$ improvement in throughput, and PD separation delivers up to 29% latency reduction. Overall, TPLA shows strong potential as a powerful and efficient replacement for MLA.

Limitation and Future Work. Although PCA demonstrates better accuracy over Hadamard transform, it has inherent limitations. Specifically, PCA concentrates most of the data’s informative content in the first few dimensions, which provide a representative summary of the global structure. In contrast, the later dimensions primarily capture negligible noise and minor variations that contribute minimally to the overall representation. Consequently, TPLA with group-partitions $g = 2$ can maintain good accuracy, but when $g > 2$, it probably fails to maintain effectiveness. By contrast, numerical-value balancing via orthogonal transforms, particularly the Hadamard transform, tends to be more effective when partitioning into multiple groups. Empirically, inserting a Hadamard transform into the RMSNorm slicing part yields almost no accuracy degradation. In future work, we will design and evaluate optimized Hadamard-like orthogonal matrices to balance softmax slicing, thereby improving both robustness and scalability. One advantage of TPLA is that it can directly inherit MLA checkpoints, but this also introduces some conversion errors. Our experiments fully validate TPLA’s expressive capacity and speed advantages. In future work, we will post-pretrain DeepSeek-V3, or train a TPLA-based model from scratch, to further demonstrate TPLA’s excellent expressiveness.

Acknowledgments

This work is supported by the National Key R&D Program of China (2022ZD0160300), Center of Excellence, Peking University, and CCF-Tencent Rhino-Bird Open Research Fund.

References

- [1] AI@Meta. 2024. Llama 3 Model Card. https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md
- [2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245* (2023).
- [3] Anthropic. 2024. Claude 3.5 Sonnet. <https://www.anthropic.com/news/claude-3-5-sonnet>
- [4] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. 2023. Longbench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508* (2023).
- [5] Loubna Ben Allal, Anton Lozhkov, Guilherme Penedo, Thomas Wolf, and Leandro von Werra. 2024. SmolLM-Corpus. (2024). <https://huggingface.co/datasets/HuggingFaceTB/SmolLM-corpus>
- [6] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. 2020. PIQA: Reasoning about Physical Commonsense in Natural Language. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*. AAAI Press, 7432–7439. doi:10.1609/AAAI.V34I05.6239
- [7] Lin Bokai, Zeng Zihao, Xiao Zipeng, Kou Siqu, Hou Tianqi, Gao Xiaofeng, Zhang Hao, and Deng Zhijie. 2024. MatryoshkaKV: Adaptive KV Compression via Trainable Orthogonal Projection. *arXiv preprint arXiv:2410.14731* (2024). <https://www.arxiv.org/abs/2410.14731>
- [8] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [9] Chang Chi-Chih, Lin Chien-Yu, Akhauri Yash, Lin Wei-Cheng, Wu Kai-Chiang, Ceze Luis, and Abdelfattah Mohamed, S. 2025. xKV: Cross-Layer SVD for KV-Cache Compression. *arXiv preprint arXiv:2503.18893* (2025). <https://www.arxiv.org/abs/2503.18893>
- [10] Chang Chi-Chih, Lin Wei-Cheng, Lin Chien-Yu, Chen Chong-Yan, Hu Yu-Fang, Wang Pei-Shuo, Huang Ning-Chi, Ceze Luis, Abdelfattah Mohamed, S., and Wu and, Kai-Chiang. 2024. Palu: Compressing KV-Cache with Low-Rank Projection. *arXiv preprint arXiv:2407.21118* (2024). <https://www.arxiv.org/abs/2407.21118>
- [11] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. *CoRR abs/1803.05457* (2018). [arXiv:1803.05457](http://arxiv.org/abs/1803.05457) <http://arxiv.org/abs/1803.05457>
- [12] Hooper Coleman, Kim Sehoon, Mohammadzadeh Hiva, Mahoney Michael, W., Shao Yakun, Sophia, Keutzer Kurt, and Gholami Amir. 2024. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. *arXiv preprint arXiv:2401.18079* (2024). <https://www.arxiv.org/abs/2401.18079>
- [13] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, Marc’aurilio Ranzato, Andrew Senior, Paul Tucker, Ke Yang, et al. 2012. Large scale distributed deep networks. *Advances in neural information processing systems* 25 (2012).
- [14] DeepSeek-AI. 2024. DeepSeek LLM: Scaling Open-Source Language Models with Longtermism. *CoRR abs/2401.02954* (2024). <https://doi.org/10.48550/arXiv.2401.02954>
- [15] DeepSeek-AI. 2024. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. *CoRR abs/2405.04434* (2024). <https://doi.org/10.48550/arXiv.2405.04434>
- [16] Weishu Deng, Yujie Yang, Peiran Du, Lingfeng Xiang, Zhen Lin, Chen Zhong, Song Jiang, Hui Lu, and Jia Rao. 2025. HGCA: Hybrid GPU-CPU

- Attention for Long Context LLM Inference. *arXiv:2507.03153* [cs.LG] <https://arxiv.org/abs/2507.03153>
- [17] Yao Dingyu, Shen Bowen, Lin Zheng, Liu Wei, Luan Jian, Wang Bin, and Wang Weiping. 2025. TailorKV: A Hybrid Framework for Long-Context Inference via Tailored KV Cache Optimization. *arXiv preprint arXiv:2505.19586* (2025). <https://www.arxiv.org/abs/2505.19586>
- [18] Amr Elmeleegy, Harry Kim, David Zier, Kyle Kranen, Nee-lay Shah, Ryan Olson, and Omri Kahalon. 2025. NVIDIA Dynamo, A Low-Latency Distributed Inference Framework for Scaling Reasoning AI Models. NVIDIA Developer Blog. <https://developer.nvidia.com/blog/introducing-nvidia-dynamo-a-low-latency-distributed-inference-framework-for-scaling-reasoning-ai-models> Published March 18, 2025.
- [19] Kim Han-Byul, Hoang Duc, Kundu Arnav, Samragh Mohammad, and Cho Minsik. 2025. SPD: Sync-Point Drop for efficient tensor parallelism of Large Language Models. *arXiv preprint arXiv:2502.20727* (2025). <https://www.arxiv.org/abs/2502.20727>
- [20] Yu Hao, Yang Zelan, Li Shen, Li Yong, and Wu Jianxin. 2024. Effectively Compress KV Heads for LLM. *arXiv preprint arXiv:2406.07056* (2024). <https://www.arxiv.org/abs/2406.07056>
- [21] Wu Haoyi and Tu Kewei. 2024. Layer-Condensed KV Cache for Efficient Inference of Large Language Models. *arXiv preprint arXiv:2405.10637* (2024). <https://www.arxiv.org/abs/2405.10637>
- [22] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net. <https://openreview.net/forum?id=d7KBjml3GmQ>
- [23] Cheng-Ping Hsieh, Simeng Sun, Samuel Krman, Shantanu Acharya, Dima Rekish, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: What's the Real Context Size of Your Long-Context Language Models? *arXiv:2404.06654* [cs.CL] <https://arxiv.org/abs/2404.06654>
- [24] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. 2019. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems* 32 (2019).
- [25] Lamprecht Itay, Karnieli Asaf, Hanani Yair, Giladi Niv, and Soudry Daniel. 2025. Tensor-Parallelism with Partially Synchronized Activations. *arXiv preprint arXiv:2506.19645v1* (2025). <https://www.arxiv.org/abs/2506.19645v1>
- [26] Hu Jie, Wang Shengnan, He Yutong, Gong Ping, Yi Jiawei, Zhang Juncheng, Bai Youhui, Chen Renhai, Zhang Gong, Li Cheng, and Yuan Kun. 2025. Efficient Long-Context LLM Inference via KV Cache Clustering. *arXiv preprint arXiv:2506.11418* (2025). <https://www.arxiv.org/abs/2506.11418>
- [27] Jinhyuk Lee, Anthony Chen, Zhuyun Dai, Dheeru Dua, Devendra Singh Sachan, Michael Boratko, Yi Luan, Sébastien M. R. Arnold, Vincent Perot, Siddharth Dalmia, Hexiang Hu, Xudong Lin, Panupong Pasupat, Aida Amini, Jeremy R. Cole, Sebastian Riedel, Iftikhar Naim, Ming-Wei Chang, and Kelvin Guu. 2024. Can Long-Context Language Models Subsume Retrieval, RAG, SQL, and More? *arXiv:2406.13121* [cs.CL] <https://arxiv.org/abs/2406.13121>
- [28] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [29] Oren Matanel, Hassid Michael, Yarden Nir, Adi Yossi, and Schwartz Roy. 2024. Transformers are Multi-State RNNs. *arXiv preprint arXiv:2401.06104* (2024). <https://www.arxiv.org/abs/2401.06104>
- [30] Fanxu Meng, Pingzhi Tang, Zengwei Yao, and Muhan Zhang. 2025. TransMLA: Multi-head Latent Attention Is All You Need. *arXiv preprint arXiv:2502.07864* (2025).
- [31] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer Sentinel Mixture Models. *arXiv:1609.07843* [cs.CL]
- [32] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. 2018. Can a Suit of Armor Conduct Electricity? A New Dataset for Open Book Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, October 31 - November 4, 2018*, Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (Eds.). Association for Computational Linguistics, 2381–2391. doi:10.18653/V1/D18-1260
- [33] Zhang Muru, Mishra Mayank, Zhou Zhongzhu, Brandon William, Wang Jue, Kim Yoon, Ragan-Kelley Jonathan, Song Shuaiwen, Leon, Athiwaratkun Ben, and Dao Tri. 2025. Ladder-residual: parallelism-aware architecture for accelerating large model inference with communication overlapping. *arXiv preprint arXiv:2501.06589* (2025). <https://www.arxiv.org/abs/2501.06589>
- [34] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized pipeline parallelism for DNN training. In *Proceedings of the 27th ACM symposium on operating systems principles*. 1–15.
- [35] OpenAI. 2024. Hello GPT-4o. <https://openai.com/index/hello-gpt-4o/>
- [36] Fu Qichen, Cho Minsik, Merth Thomas, Mehta Sachin, Rastegari Mohammad, and Najibi Mahyar. 2024. LazyLLM: Dynamic Token Pruning for Efficient Long Context LLM Inference. *arXiv preprint arXiv:2407.14057* (2024). <https://www.arxiv.org/abs/2407.14057>
- [37] Xu Qifan, Li Shenggui, Gong Chaoyu, and You Yang. 2021. An Efficient 2D Method for Training Super-Large Deep Learning Models. *arXiv preprint arXiv:2104.05343* (2021). <https://www.arxiv.org/abs/2104.05343>
- [38] Li Qingyuan, Zhang Bo, Ye Liang, Zhang Yifan, Wu Wei, Sun Yeru, Ma Lin, and Xie Yuchen. 2024. Flash Communication: Reducing Tensor Parallelization Bottleneck for Fast Large Language Model Inference. *arXiv preprint arXiv:2412.04964* (2024). <https://www.arxiv.org/abs/2412.04964>
- [39] Alec Radford. 2018. Improving language understanding by generative pre-training. (2018).
- [40] Zhang Rongzhi, Wang Kuang, Liu Liyuan, Wang Shuohang, Cheng Hao, Zhang Chao, and Shen Yelong. 2024. LoRC: Low-Rank Compression for LLMs KV Cache with a Progressive Compression Strategy. *arXiv preprint arXiv:2410.03111* (2024). <https://www.arxiv.org/abs/2410.03111>
- [41] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2021. WinoGrande: an adversarial winograd schema challenge at scale. *Commun. ACM* 64, 9 (2021), 99–106. doi:10.1145/3474381
- [42] Alexander Sergeev and Mike Del Balso. 2018. Horovod: fast and easy distributed deep learning in TensorFlow. *arXiv preprint arXiv:1802.05799* (2018).
- [43] Smith Shaden, Patwary Mostofa, Norick Brandon, LeGresley Patrick, Rajbhandari Samyam, Casper Jared, Liu Zhun, Prabhumoye Shrimai, Zerveas George, Korthikanti Vijay, Zhang Elton, Child Rewon, Aminabadi Reza, Yazdani, Bernauer Julie, Song Xia, Shoenybi Mohammad, He Yuxiong, Houston Michael, Tiwary Saurabh, and Catanzaro and, Bryan. 2022. Using DeepSpeed and Megatron to Train Megatron-Turing NLG 530B, A Large-Scale Generative Language Model. *arXiv preprint arXiv:2201.11990* (2022). <https://www.arxiv.org/abs/2201.11990>
- [44] Rajput Shashank, Sheng Ying, Owen Sean, and Chiley Vitaliy. 2024. Inference-Friendly Models With MixAttention. *arXiv preprint arXiv:2409.15012* (2024). <https://www.arxiv.org/abs/2409.15012>
- [45] Li Shenggui, Xue Fuzhao, Baranwal Chaitanya, Li Yongbin, and You Yang. 2021. Sequence Parallelism: Long Sequence Training from System Perspective. *arXiv preprint arXiv:2105.13120* (2021). <https://www.arxiv.org/abs/2105.13120>
- [46] Dong Shichen, Cheng Wen, Qin Jiayu, and Wang Wei. 2024. QAQ: Quality Adaptive Quantization for LLM KV Cache. *arXiv preprint arXiv:2403.04643* (2024). <https://www.arxiv.org/abs/2403.04643>

- [47] Mohammad Shoneybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).
- [48] Ge Suyu, Zhang Yunan, Liu Liyuan, Zhang Minjia, Han Jiawei, and Gao Jianfeng. 2023. Model Tells You What to Discard: Adaptive KV Cache Compression for LLMs. *arXiv preprint arXiv:2310.01801* (2023). <https://www.arxiv.org/abs/2310.01801>
- [49] Gemini Team, Petko Georgiev, Ving Ian Lei, Ryan Burnell, Libin Bai, Anmol Gulati, Garrett Tanzer, Damien Vincent, Zhufeng Pan, Shibo Wang, et al. 2024. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530* (2024).
- [50] Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijie Chen, Yanru Chen, Yuankun Chen, Yutian Chen, et al. 2025. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534* (2025).
- [51] Zadouri Ted, Strauss Hubert, and Dao Tri. 2025. Hardware-Efficient Attention for Fast Decoding. *arXiv preprint arXiv:2505.21487v1* (2025). <https://www.arxiv.org/abs/2505.21487v1>
- [52] Munkhdalai Tsendsuren and Siddharth Gopal Manaal, Faruqi. 2024. Leave No Context Behind: Efficient Infinite Context Transformers with Infini-attention. *arXiv preprint arXiv:2404.07143* (2024). <https://www.arxiv.org/abs/2404.07143>
- [53] Brandon William, Mishra Mayank, Nrusimha Aniruddha, Panda Rameswar, and Kelly Jonathan, Ragan. 2024. Reducing Transformer Key-Value Cache Size with Cross-Layer Attention. *arXiv preprint arXiv:2405.12981* (2024). <https://www.arxiv.org/abs/2405.12981>
- [54] Zhou Xiabin, Wang Wenbin, Zeng Minyan, Guo Jiaxian, Liu Xuebo, Shen Li, Zhang Min, and Ding Liang. 2024. DynamicKV: Task-Aware Adaptive KV Cache Compression for Long Context LLMs. *arXiv preprint arXiv:2412.14838* (2024). <https://www.arxiv.org/abs/2412.14838>
- [55] Lin Xiaolin, Wang Jingcun, Kondratyeva Olga, Shi Yiyu, Li Bing, and Zhang Grace, Li. 2025. CompressKV: Semantic Retrieval Heads Know What Tokens are Not Important Before Generation. *arXiv preprint arXiv:2508.02401v1* (2025). <https://www.arxiv.org/abs/2508.02401v1>
- [56] Liu Xin, Liu Pei, and Tang Guoming. 2025. ZSMerge: Zero-Shot KV Cache Compression for Memory-Efficient Long-Context LLMs. *arXiv preprint arXiv:2503.10714* (2025). <https://www.arxiv.org/abs/2503.10714>
- [57] Amy Yang, Jingyi Yang, Aya Ibrahim, Xinfeng Xie, Bangsheng Tang, Grigory Sizov, Jeremy Reizenstein, Jongsoo Park, and Jianyu Huang. 2025. Context Parallelism for Scalable Million-Token Inference. *arXiv:2411.01783* [cs.DC] <https://arxiv.org/abs/2411.01783>
- [58] Yang Yifei, Cao Zouying, Chen Qiguang, Qin Libo, Yang Dongjie, Zhao Hai, and Chen Zhi. 2024. KVSharer: Efficient Inference via Layer-Wise Dissimilar KV Cache Sharing. *arXiv preprint arXiv:2410.18517* (2024). <https://www.arxiv.org/abs/2410.18517>
- [59] Wu You, Wu Haoyi, and Tu Kewei. 2024. A Systematic Study of Cross-Layer KV Sharing for Efficient LLM Inference. *arXiv preprint arXiv:2410.14442* (2024). <https://www.arxiv.org/abs/2410.14442>
- [60] Li Yuhong, Huang Yingbing, Yang Bowen, Venkatesh Bharat, Locatelli Acyr, Ye Hanchen, Cai Tianle, Lewis Patrick, and Chen Deming. 2024. SnapKV: LLM Knows What You are Looking for Before Generation. *arXiv preprint arXiv:2404.14469* (2024). <https://www.arxiv.org/abs/2404.14469>
- [61] Sungmin Yun, Seonyong Park, Hwayong Nam, Younjo Lee, Gunjun Lee, Kwanhee Kyung, Sangpyo Kim, Nam Sung Kim, Jongmin Kim, Hyungyo Kim, Juhwan Cho, Seungmin Baek, and Jung Ho Ahn. 2025. The New LLM Bottleneck: A Systems Perspective on Latent Attention and Mixture-of-Experts. *arXiv:2507.15465* [cs.AR] <https://arxiv.org/abs/2507.15465>
- [62] Kawakibi Zuhri Zayd, Muhammad, Adilazuarda Muhammad, Farid, Purwarianti Ayu, and Aji Alham, Fikri. 2024. MLKV: Multi-Layer Key-Value Heads for Memory Efficient Transformer Decoding. *arXiv preprint arXiv:2406.09297* (2024). <https://www.arxiv.org/abs/2406.09297>
- [63] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. HellaSwag: Can a Machine Really Finish Your Sentence?. In *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28– August 2, 2019, Volume 1: Long Papers*, Anna Korhonen, David R. Traum, and Lluís Màrquez (Eds.). Association for Computational Linguistics, 4791–4800. doi:10.18653/V1/P19-1472
- [64] Juntao Zhao, Jiuru Li, and Chuan Wu. 2025. Sandwich: Separating Prefill-Decode Compilation for Efficient CPU LLM Serving. *arXiv preprint arXiv:2507.18454* (2025).
- [65] Wang Zheng, Jin Boxiao, Yu Zhongzhi, and Zhang Minjia. 2024. Model Tells You Where to Merge: Adaptive KV Cache Merging for LLMs on Long-Context Tasks. *arXiv preprint arXiv:2407.08454* (2024). <https://www.arxiv.org/abs/2407.08454>
- [66] Bian Zhengda, Xu Qifan, Wang Boxiang, and You Yang. 2021. Maximizing Parallelism in Distributed Training for Huge Neural Networks. *arXiv preprint arXiv:2105.14450* (2021). <https://www.arxiv.org/abs/2105.14450>
- [67] Zhang Zhenyu, Sheng Ying, Zhou Tianyi, Chen Tianlong, Zheng Lianmin, Cai Ruisi, Song Zhao, Tian Yuandong, Ré Christopher, Barrett Clark, Wang Zhiyang, and Chen Beidi. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. *arXiv preprint arXiv:2306.14048* (2023). <https://www.arxiv.org/abs/2306.14048>
- [68] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 193–210.
- [69] Liu Zirui, Yuan Jiayi, Jin Hongye, Zhong Shaochen, Xu Zhaozhao, Braverman Vladimir, Chen Beidi, and Hu Xia. 2024. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. *arXiv preprint arXiv:2402.02750* (2024). <https://www.arxiv.org/abs/2402.02750>
- [70] Wang Zongwu, Xu Peng, Liu Fangxin, Hu Yiwei, Sun Qingxiao, Li Gezi, Li Cheng, Wang Xuan, Jiang Li, and Guan Haibing. 2025. MILLION: Mastering Long-Context LLM Inference Via Outlier-Immune KV Product Quantization. *arXiv preprint arXiv:2504.03661* (2025). <https://www.arxiv.org/abs/2504.03661>

A Additional Long-Context Benchmark Evaluation

In addition to the commonsense reasoning tasks and Long-Bench reported in the main text (Table 1 and Table 2), we further evaluate on two additional long-context benchmarks: LOFT [27] and RULER [23].

LOFT evaluates in-context retrieval and reasoning across multiple tasks with sequence lengths ranging from 32K to 1M tokens. Since DeepSeek-V2-Lite is not optimized for extremely long contexts, its performance on LOFT retrieval, RAG, and SQL tasks is very low (approximately 0–0.15). We therefore focus on the 32K ICL tasks, where the model behaves reliably. In this setting, MLA achieves an accuracy of 0.345, and **TPLA (pd sep.)** matches it at 0.345, further confirming that TPLA preserves accuracy.

RULER provides synthetic long-context tasks with configurable sequence length and task complexity, measuring capabilities beyond simple in-context recall. We evaluate DeepSeek-V2-Lite and its TPLA variants on all 13 RULER

tasks across four context lengths (4,096 / 8,192 / 16,384 / 32,684 tokens). Table 3 reports the average score across the 13 tasks at each context length.

Overall, the results suggest that TPLA with PD-separation better preserves performance in shorter-context decoding scenarios (e.g., LOFT ICL and the commonsense tasks in Table 1, both evaluated in a multiple-choice setting). In contrast, for long-decoding settings (e.g., long-CoT style generation), the TPLA conversion can introduce non-negligible approximation errors. As the generation length increases, such errors may accumulate, leading to a larger accuracy drop. We further observe that the aligned TPLA variant narrows the gap at shorter contexts (Table 1 and 2), but still exhibits a notable performance drop at longer contexts in RULER (Table 3). This is expected, as our lightweight alignment uses

training sequences up to 4K tokens, and thus does not directly optimize the model for substantially longer-context behaviors (e.g., 16K–32K). Developing more effective and efficient long-context alignment strategies for TPLA therefore remains an important direction for future work.

Table 3. RULER accuracy under different context lengths.

Model	4K	8K	16K	32K
DeepSeek-V2-Lite	52.50	42.88	46.97	37.92
- TPLA (<i>pd sep.</i>)	21.82	15.38	16.58	17.41
- TPLA (<i>align</i>)	47.58	42.53	35.47	26.87